BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
SPECIALIZATION COMPUTER SCIENCE IN ENGLISH

DIPLOMA THESIS

Implementing Rollback Netcode and Deterministic Physics in a Unity-Based 2D Platform Fighter

Supervisor
Lect. Zsigmond Imre

Author
Sabaceag Marius-Vasile

2026

ABSTRACT

Abstract: un rezumat în limba engleză cu prezentarea, pe scurt, a conținutului pe capitole, punând accent pe contribuțiile proprii și originalitate

Contents

1 Introduction **1**

2 State Synchronization in Multiplayer Games **2**
2.1 Network Architectures in Multiplayer Games 2
2.2 Common Synchronization Approaches 3
2.2.1 Delay-Based State Synchronization 3
2.2.2 Rollback-based Synchronization 3
2.3 Technical Prerequisites for Rollback Integration 4

3 Designing a Deterministic Physics Engine **5**
3.1 The Determinism Problem in Game Engines 5
3.2 Core Physics Data Structures 6
3.2.1 Collider Geometries 6
3.2.2 Dynamic Body Representation 7
3.3 Collision Detection and Collision Resolution 7
3.3.1 Broad-phase Collision Detection 8
3.3.2 Narrow-phase Pairwise Checks 9

4 System Architecture and Layer Decoupling **10**
4.1 High-Level Architectural Overview 10
4.1.1 Layer Definitions and Responsibilities 11
4.2 Decoupling Logic from Presentation 11
4.2.1 The Manual Ticking System 12
4.2.2 Visual State Synchronization 12

5 Gameplay and Combat System Implementation **13**
5.1 Deterministic Character Controllers 13
5.2 The Combat Engine 13
5.3 Animation Synchronization 13

6 Implementation of the Rollback Netcode **14**

6.1 Network Communication Protocol 14
6.2 Game State Management 14
6.3 The Game Manager and Rollback 14

7 Conclusions **15**

Bibliography **16**

Chapter 1

Introduction

Introducere: obiectivele lucrării și descrierea succintă a capitolelor, prezentarea temei, prezentarea contribuției proprii, respectiv a rezultatelor originale și menționarea (dacă este cazul) a sesiunii de comunicări unde a fost prezentată sau a revistei unde a fost publicată.

Chapter 2

State Synchronization in Multiplayer Games

2.1 Network Architectures in Multiplayer Games

Multiplayer games are usually built on either a peer-to-peer or a client-server architecture, each offering distinct advantages and trade-offs between scalability, security, and latency [SLJ+24].

In a client-server architecture, the state of the game is handled by the server, which is always processing incoming data from all clients, resulting in an accurate representation of the game. However, this means that each client experiences increased latency due to the server acting as a relay [HV19].

Peer-to-peer architecture is more suitable for fighting games, as it can achieve fast response times by creating direct connections between players, but it introduces complexity in maintaining global state consistency and is often more vulnerable to distributed attacks and cheating [YK13].

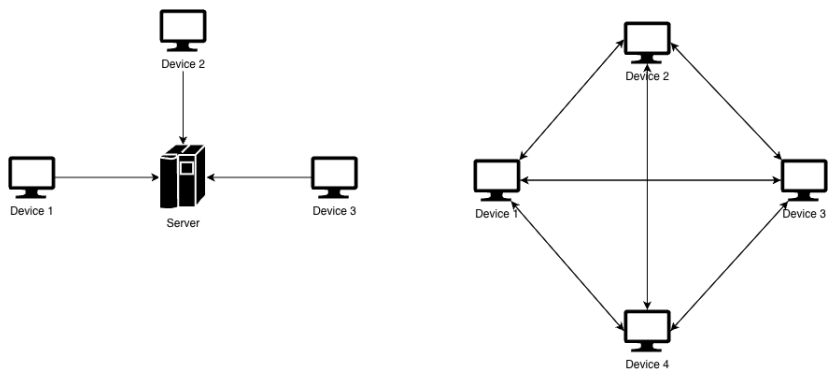

Figure 2.1: Client-Server architecture and Peer-to-Peer architecture

2.2 Common Synchronization Approaches

2.2.1 Delay-Based State Synchronization

Delay-Based networking is the traditional method of synchronizing the state of the game on a Peer-to-Peer network. It is based on the lockstep mechanism, which is a synchronization method where all players' actions are processed simultaneously before updating the game state [Vii25]. This will result in the game state being theoretically identical across everyone, because each client will wait to receive the input from every other client before the simulation advances to the next frame.

However, the speed of the game is limited by the slowest connection, since the other clients must wait for the client with the worst connection to send their input to everyone else. This is a major problem for fighting games, where responsiveness and consistency is important, because the clients will experience inconsistencies with the responsiveness of their inputs as the network latency fluctuated. [HV19]

2.2.2 Rollback-based Synchronization

Rollback-based approach is the alternative way to handle multiplayer synchronization. While it's implementation is more complex, if done correctly it can essentially replicate the feeling of 'local' game-play. The key difference between these two approaches lies in what is happening before we get the input from the other clients.

To simplify the explanation of this approach, We will consider that there are only two clients playing the game. When the input for the remote client is missing, the delay-based approach will pause and wait until the input arrives, which as discussed above will cause variable latency. The rollback netcode will never wait for the missing input from the other client. Instead, rollback continues to run the game as usual, which results in all inputs from the local player to be processed instantly [Pus19]. When the remote input arrives a few frames later, the rollback netcode will fix the mistake by going back to the frame where the remote input was executed, and then simulating all the frames using the corrected input of the remote player, until we reach the current in-game frame. This will usually result in the remote player's animations skipping some of their start-up frames. Two important details of this to note are:

- The simulation of the previous frames is happening in the background. The local player does not see all of these frames, instead the game will render only the final frame.
- The local player's input will always be consistent, even if a rollback occurs

during it.

This approach can be improved by trying to predict the input of the remote player. If we assume the remote player is not doing anything when their input is missing, then the visual state shown to the player right before a rollback would be wildly incorrect, and it is a flawed assumption, because players rarely are not doing anything [Pus19].

We can instead assume the current input is identical to the most recently received input from the remote player. This prediction algorithm is also used in GGPO (Good Game, Peace Out), which is a rollback networking framework used in other fighting games, developed by Tony Cannon specifically to attempt to hide network lag in fighting games [HV19].

2.3 Technical Prerequisites for Rollback Integration

While rollback netcode performs better and it is more desirable than the delay-based netcode, implementing rollback may pose a challenge, because a game must meet the following prerequisites in order to support rollback:

The Determinism Requirement

The most fundamental requirement for implementing rollback on a game is that it must be deterministic; i.e. if given the same input, it will always produce the same outcome, regardless of the machine used. This is important because the rollback netcode synchronizes only the client inputs, so the game simulation will run for each client, and any difference between the results will eventually lead to completely different game states.

Computational Overhead

Since we need to perform a rollback as fast as possible, preferably within a single frame, we need to manage the previous states and physics simulation as efficiently as possible [HV19]. We need to periodically save the entire game state, such that a previous state can be restored if needed, which may consume substantial memory.

Chapter 3

Designing a Deterministic Physics Engine

3.1 The Determinism Problem in Game Engines

As mentioned in the previous chapter, the architectural requirement for a peer-to-peer rollback system is entirely based on the concept of determinism. A simulation is deterministic if, given identical initial conditions and a synchronized sequence of inputs, it goes through identical states and has the same result on all clients, regardless of the compiler, operating system, or hardware.

This requirement poses a significant challenge for game development in Unity, because according to an article from Unity's official Support, the 2D physics engine is not deterministic across different platforms. The reason for this is that depending on the compiler or processor architecture, the implementation of floating-point arithmetic can differ, which may affect the results of the physics simulation. [Tec24]

To combat this, you must either find and use another physics engine, or create one from scratch. This physics engine cannot use floating points, and instead fixed-point numbers should be used, as they offer a consistent way of representing decimal values. [Vii25].

In fixed-point arithmetic, real numbers are represented as integers, where a number of bits is used to represent the whole number, and the rest of the bits are for representing the fractional part. In the physics engine we are going to use the library Unity.Mathematics.FixedPoint [Man19], that is an extension of the library FixedMath.NET, a C# fixed-point library that represents real numbers on a 64 bit integer, where 31 bits represent the whole part of the number, and 32 bits are reserved

for the fractional part. Unity.Mathematics.FixedPoint implements deterministic versions for the operations found in Unity's math library Unity.Mathematics, and defines structures for the objects used in Unity, such as fp2, which is the deterministic version of Vector2, representing a 2D point.

3.2 Core Physics Data Structures

To transition from a standard physics engine to a custom one, we need to redesign its core components and their interaction. The components that we are going to need are Colliders and Dynamic Bodies.

3.2.1 Collider Geometries

Inside Unity's documentation about its physics engine, a collider is defined as the shape given to an object for the pourpose of physical collisions. And they can be static or dynamic, depending if they have attached to them a body or not. [Uni24a] We are also going to check for collisions between hitboxes and hurtboxes, which are colliders that will not physically interact with the other colliders.

The colliders inside the physics engine are going to be represented by simple geometric shapes. Each shape will be defined by their position (the center of the shape), which is a 2D point in space, plus other specific properties. We will define the following collider shapes in the physics engine: Circle, Box and Capsule.

The box collider is represented as an Axis Alligned Bounding Box (AABB - Add reference here to 3.3.1). It is defined by its extends, which are half of the box's length and width.

We use the circle collider due to its inexpensive intersection test and the fact that they cannot be rotated, so transforming the circle is trivial, as you only need to translate its position. [Eri04] It is defined by the radius r.

By far the most complex shape we will use as a collider, the capsule, according to Ericson, is the result of sweeping a sphere (circle for 2D physics) of radius r along a line segment. [Eri04] It is defined by the radius r and half of the length of the line segment. We are going to use capsules for representing the character's hurtboxes, so the capsule collider may also be rotated.

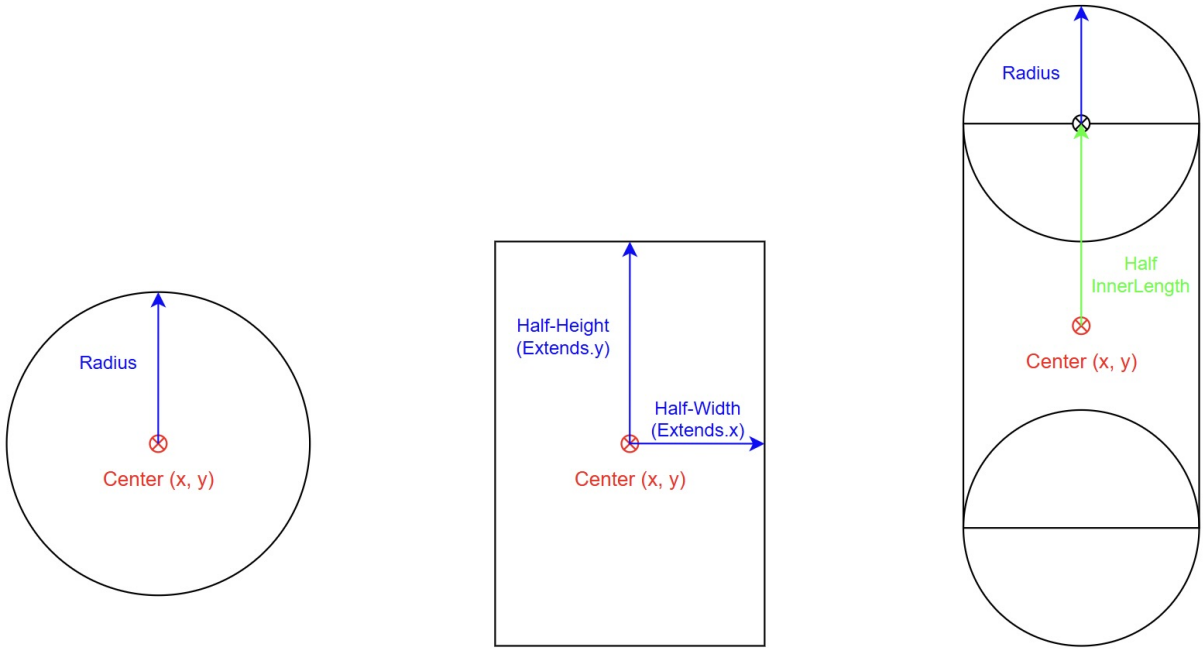

Figure 3.1: The different collider shapes and their representation

3.2.2 Dynamic Body Representation

Inside Unity's physics engine, you use Rigidbodies to objects to control them with the physics engine, and to be able to move colliders and make them interact with each other. [Uni24b] In the physics engine we are going to define a DynamicBody, that will allow the characters to collide with the environment. Inside the DynamicBody we are going to define the following properties:

- Collider - the collider that is going to interact with the other physics objects
- Position - the collider's position in world space
- Velocity - the speed and direction of the body in space
- External velocity - velocity that is applied from other objects (in our case, attack's knockback). This velocity will decay over time.

3.3 Collision Detection and Collision Resolution

Collision detection is one of the most important features of the physics engine, since it is what allows physics objects to interact with one another. Calculating these collision may impact performance if they are not implemented properly. To detect if some objects are colliding, we need to check for every collider if they are intersecting with every other collider, which without any optimizations, will result in performing $O(N^2)$ operations to check the collisions between N objects. [TCPS17]

Because we need to support performing multiple simulations within a few milliseconds to accommodate the re-simulations resulting from the rollback synchronization, we need to optimize the collision detection. To speed up this process, we need to reduce the number of pairs of objects that are being checked for collisions. This reduction is done by separating the collision detection of multiple objects in two phases: the broad phase and the narrow phase. [Eri04]

3.3.1 Broad-phase Collision Detection

The broad phase is a culling phase that detects the pair of objects that are definitely not colliding. To achieve this, we are going to use Axis-Aligned Bounding Boxes (AABBs), which is defined as a box in 2D dimension that has all of its edges parallel with the axes of the coordinate system they are in (they cannot be rotated).

The overlap check is very fast, because two AABBs overlap if and only if they overlap on both axes. [Eri04] This means the boxes are not colliding if for either collider their left segment is on the right side of the other collider's right segment, or their bottom segment is above the other's top segment (how it happens between object C and either of the other 2 objects) 3.3.1

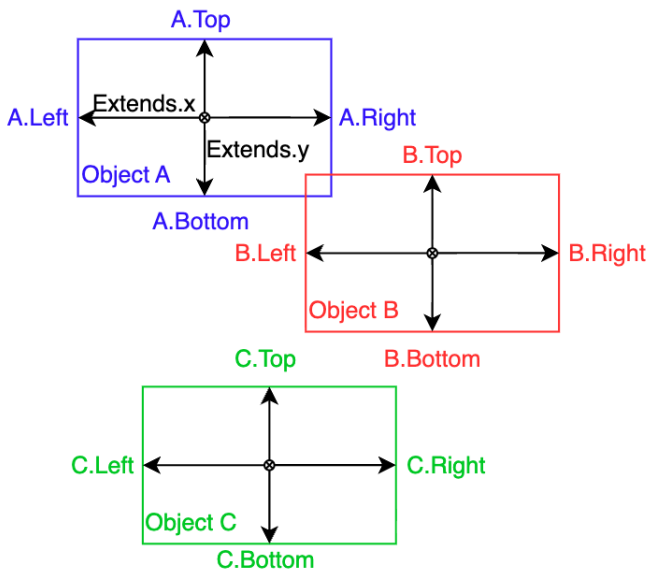

Figure 3.2: The different collider shapes and their representation

For the broad phase we can simply take every collider, regardless of their shape, and envelop the colliders by an AABB that defines its maximum extent along the X and Y axes, then do pair-wise checks with these AABB to find out if this pair of objects are either not colliding, or there is a possible collision between them.

3.3.2 Narrow-phase Pairwise Checks

After we find the pair of objects that may be colliding, we perform the narrow phase of the collision handling. This phase constitutes in checking every pair from the previous phase that maybe were colliding, and check if their shapes really collide or not. We are going to cover how to check for intersections between the following shape pairs: Circle - Circle, Circle - Box, Capsule - Circle and Capsule - Box. The Box - Box check is equivalent to the AABB collision check, and we are not going to use anywhere the Capsule - Capsule collision, so it is not going to be implemented.

Circle-Circle Collision check

The collision check between two circles is very simple: we calculate sum of the circle radii, and then we compare it against the Euclidean distance between the centers of the circles. To optimize this check, we compare the squared distances instead, as the square root operation is expensive, and in this case not useful. [Eri04]

Circle-Box Collision check

Checking if a Circle and an AABB are intersecting is done by getting the point from the box that is the closest to the center of the circle, and comparing the distance between this point and the center of the circle against the radius of the circle (and like in the previous case, we compare the squared values). Finding the closest point of an AABB to a point P is as simple as clamping P to the bounds of the Box on a component-wise basis (clamp the horizontal coordinate of P to the horizontal bounds, and then clamp the vertical coordinate of P to the vertical bounds) [Eri04]

Closest point to a line segment

As mentioned in the subsection describing collider shapes 3.2.1, the capsule is essentially a circle that is swept along a segment, so for the capsule collision checks against the other shapes we are going to find the closest circle inside the capsule to the other collider, and then check if this circle is colliding with the other shape. To do this we need to calculate the closest point from the other collider's center and the capsule's segment, which will be the center of the closest circle to the collider. [Eri04]

If the capsule is not rotated (axis alligned), the center's horizontal position is equal to the capsule's horizontal value (because the capsule is axis alligned, all the points on the segment will have the same horizontal value), and the vertical value can be found by clamping the shape's vertical position to the capsule's segment.

Chapter 4

System Architecture and Layer Decoupling

The implementation of rollback netcode and deterministic physics within a Unity-based environment necessitates a fundamental departure from the engine's native workflows. Traditional Unity development often encourages a monolithic approach where gameplay logic, physics, and visual representation are tightly coupled within the `MonoBehaviour` lifecycle. However, to facilitate the rapid state rewinding and resimulation required for rollback, a rigorous architectural decoupling is mandatory. This chapter details the four-layer architectural framework designed to isolate the deterministic simulation from the non-deterministic rendering and input systems of the host engine, ensuring bit-identical synchronization across networked clients.

4.1 High-Level Architectural Overview

The architectural design of the platform fighter is governed by the requirement for total state reproducibility. Rollback netcode operates on an optimistic execution model: the local client processes inputs immediately and renders the resulting state without waiting for confirmation from remote peers [BDST25]. When remote inputs eventually arrive, they are compared against the local prediction. If a discrepancy is found, the system must revert to the last known-good state and re-execute all intervening frames [BDST25, Pus19].

To achieve this at the 60 frames-per-second (FPS) requirement of a fighting game, the game state must be entirely distinct from Unity's `GameObject` hierarchy. Unity's standard systems—including its PhysX-based physics engine, its floating-point-heavy `Transform` system, and its internal clock loops—introduce subtle non-determinism

that accumulates over time [CGM+22, Tec24]. Consequently, the system architecture treats Unity strictly as a rendering and input-capture API, while the custom simulation engine maintains an independent reality.

4.1.1 Layer Definitions and Responsibilities

The system is divided into four hierarchical layers, each with a narrow scope of operation and a clear data boundary. This unidirectional dependency ensures that data flows from the static logic to the dynamic visuals without reverse coupling.

Layer 1: The State (Data Layer) The State layer is the "source of truth" for the simulation. It is composed of pure C# `structs` that represent the entire mutable world at a given tick. To avoid Garbage Collection (GC) spikes—which would be triggered by the frequent copying of objects during rollbacks—this layer contains zero reference types. It uses fixed-point numbers for spatial data and integer bitmasks for state tracking [Man19].

Layer 2: The Simulation (Logic Layer) This layer contains the mathematical heart of the project. It is comprised of stateless, static classes—primarily the `CombatEngine` and `PhysicsEngine`. These systems operate on the Piecewise Deterministic (PWD) assumption: if the state at tick t and the input for tick t are known, the state at tick $t + 1$ can be calculated with 100% precision [Vii25].

Layer 3: The View (Presentation & Factory Layer) The View layer serves as the bridge between the custom simulation and the Unity Engine. It reads the current state from Layer 1 and updates visual components. A critical component is the "Factory" system, which translates Unity-authored data (like scene platforms) into deterministic formats used by Layer 1 [Llo03].

Layer 4: The Core (Orchestration Layer) The Core Layer acts as the master scheduler. It owns the simulation "time," capturing user input and feeding it into the manual ticking system while maintaining the Ring Buffer of past snapshots for rollbacks [BDST25, SO25].

4.2 Decoupling Logic from Presentation

The core challenge of this architecture is the "revocation of Unity's autonomy." Unity is relegated to a rendering slave, separating the *Game State* (logical reality)

from the *Visual State* (interpolated representation). This separation is vital for networked physics, where the simulation might predict multiple frames ahead while the visuals remain smooth for the user [CKFJ03, HV19].

4.2.1 The Manual Ticking System

To maintain synchronicity, the game implements a manual ticking system using a time accumulator to ensure the logic ticks at exactly 60 Hz regardless of the rendering speed [CGM+22]. The logic of the manual ticking system is defined by:

$$A = A + \Delta t_{render} \tag{4.1}$$

While $A \geq \Delta t_{fixed}$, the system executes:

$$ExecuteSimulation(S_{current}, I_{local}, I_{remote}) \tag{4.2}$$

This allows for "Catch-up Logic," where multiple simulation ticks execute in a single render frame if the game lags, which is the cornerstone of the rollback mechanism [BDST25].

4.2.2 Visual State Synchronization

The View layer must synchronize visual states without appearing "choppy." This is achieved through state interpolation. The system stores a `PreviousState` and a `TargetState` and calculates an alpha value (α) based on the remaining time in the accumulator to perform linear interpolation [JO26].

$$P_{visual} = Lerp(S_{prev}.pos, S_{target}.pos, \alpha) \tag{4.3}$$

Additionally, to ensure animations do not drift during rollbacks, the `Animator.speed` is set to 0, and the Core manually snaps the `normalizedTime` on every frame based on the current logic tick [SLJ+24].

Chapter 5

Gameplay and Combat System Implementation

5.1 Deterministic Character Controllers

5.2 The Combat Engine

5.3 Animation Synchronization

Chapter 6

Implementation of the Rollback Netcode

6.1 Network Communication Protocol

6.2 Game State Management

6.3 The Game Manager and Rollback

Chapter 7

Conclusions

Concluzii ...

Bibliography

[BDST25] Yérom-David Bromberg, Jérémie Decouchant, Manon Sourisseau, and François Taïani. Formalizing rollback netcodes for robust and real-time client-server architectures. In *29th International Conference on Principles of Distributed Systems (OPODIS 2025)*, 2025.

[CGM+22] Greg Chance, Abanoub Ghobrial, Kevin McAreavey, Séverin Lemaignan, Tony Pipe, and Kerstin Eder. On determinism of game engines used for simulation-based autonomous vehicle verification. *arXiv*, 2104.06262, 2022.

[CKFJ03] Eric Cronin, Anthony R. Kurc, Burton Filstrup, and Sugih Jamin. An efficient synchronization mechanism for mirrored game architectures. *Kluwer Academic Publishers*, 2003.

[Eri04] Christer Ericson. *Real-Time Collision Detection*. Morgan Kaufmann Series in Interactive 3D Technology. CRC Press, 2004.

[HV19] Martin Huynh and Fernando Valarino. An analysis of continuous consistency models in real time peer-to-peer fighting games. 2019.

[JO26] Longxiang Jiao and Others. Bridging the gap between micro-scale traffic simulation and 4d digital cityscapes. 2026.

[Llo03] Noel Llopis. Delicious data baking. *Game Developer Magazine*, 2003.

[Man19] Daniel Mansson. Unity.mathematics.fixedpoint extension library, 2019.

[Pus19] Ricky Pusch. Explaining how fighting games use delay-based and rollback netcode. *Ars Technica*, page 32, 2019.

[SLJ+24] Michael Siebenmann, Mathieu Lutfallah, Dominic Jetter, Christian Hirt, and Andreas Kunz. Investigation into recording, replay and simulation of interactions in virtual reality. In *Proceedings of the 2024 8th Interna-*

tional Conference on Virtual and Augmented Reality Simulations, pages 13–21, 2024.

[SO25] Antônio Sousa and Others. A performance analysis of system rollback techniques. *ResearchGate*, 2025.

[TCPS17] Michael Tanaya, Huaming Chen, Jebediah Pavleas, and Kelvin Sung. *Building a 2D Game Physics Engine*. Springer, 2017.

[Tec24] Unity Technologies. Determinism with 2d physics, 2024.

[Uni24a] Unity Technologies. *Colliders Overview*, 2024. Accessed: 2026-04-21.

[Uni24b] Unity Technologies. *Rigidbody 2D*, 2024. Accessed: 2026-04-21.

[Vii25] Henri Viitanen. Deterministic and synchronous computation between client and server in mobile games. 2025.

[YK13] Amir Yahyavi and Bettina Kemme. Peer-to-peer architectures for massively multiplayer online games: A survey. *ACM Computing Surveys (CSUR)*, 46(1):1–51, 2013.